

TONBO Phase 3 RNM:PyTorch Model Matching Report

June 2026

Document Version v0.1
SDK Version 26.05

1	Context and Description	3
1.1	Review of Mythic's Modeling Process	3
1.1.1	Accuracy Simulator (PyTorch) vs. SystemVerilog / RNM	3
	Hierarchy of Modeling Methods	3
	Mythic Modeling Process	4
2	Model Matching Discussion	6
2.1	Components Being Modeled	6
2.1.1	Signal Chain Diagram	6
2.2	AIDAC Model	6
2.2.1	Diagram	6
2.2.2	RNM vs. PyTorch Model Match Without Manufacturing Variation	6
2.2.3	RNM vs. PyTorch Model Match With Identical Manufacturing Variation	7
2.2.4	AIDAC Model Summary	8
2.3	NVM Model	9
2.4	ADC Model	9
2.4.1	Diagram	9
2.4.2	RNM vs. PyTorch Model Match Without Manufacturing Variation	9
2.4.3	RNM vs. PyTorch Model Match With Identical Manufacturing Variation	11
2.4.4	ADC Model Summary	13
3	Full ACE Model Comparison With Real Network Data	13
3.1	RNM vs. PyTorch Model Match Without Manufacturing Variation	13
3.2	RNM vs. PyTorch Model Match With Manufacturing Variation	15
3.3	Full ACE Model Comparison Summary	16
4	Conclusion	17

MYTHIC

1 Context and Description

The purpose of this document is to show that Mythic has developed SystemVerilog / RNM models of the ACE which match the PyTorch / Accuracy Simulator models to a very high degree of precision.

1.1 Review of Mythic's Modeling Process

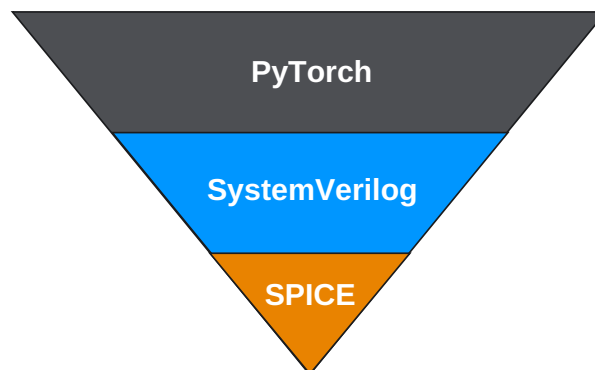
Here we will repeat some information previously described in section 4 ("Ace Modeling") from the Analog Compute Engine Operation document.

1.1.1 Accuracy Simulator (PyTorch) vs. SystemVerilog / RNM

Hierarchy of Modeling Methods

Mythic is using a hierarchy of methods to verify system and network accuracy:

Figure 1. Modeling Hierarchy



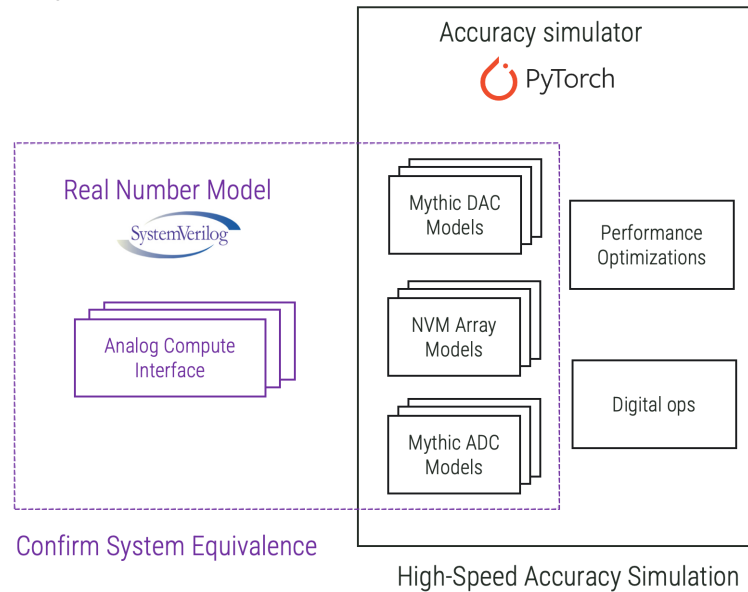
1. SPICE simulation
 - SPICE is the most accurate circuit simulation method, but it is the most computationally expensive
 - We simulate our ADCs, AIDACs, NVM, and other analog circuits with SPICE to characterize our circuit behavior and create **reference models**
 - Our reference model behavior matches SPICE simulations
 - SPICE is capable of simulating thousands of devices, but not running a full computation of the entire ACE
2. SystemVerilog / Real Number Modeling
 - Real Number Modeling is a Verilog enhancement that allows digital simulators to model analog behavior with real numbers (and not just binary or logical values).
 - RNMs allow mixed-signal designs to be verified with the same powerful and large-scale methods as Digital Verification
 - Having a SystemVerilog / RNM model of the ACE allows Mythic to verify the ACE the same way all digital processors are verified, and be verified with all of the surrounding digital circuits in the system
 - Mythic's Real Number Models of the analog components are built with the same **reference models created by SPICE simulation**
 - Digital Verification is capable of simulating an entire ACE, and small simulations of our entire chip, but not an entire network

MYTHIC

3. PyTorch

- PyTorch is a popular and open-source machine learning framework package in Python
- PyTorch is how Mythic creates, trains, and analyzes performance of our networks
- PyTorch is how to simulate an entire network and compare accuracy using the **reference models for computation**

Figure 2. Verilog and Accuracy Simulators Use The Same Reference Models



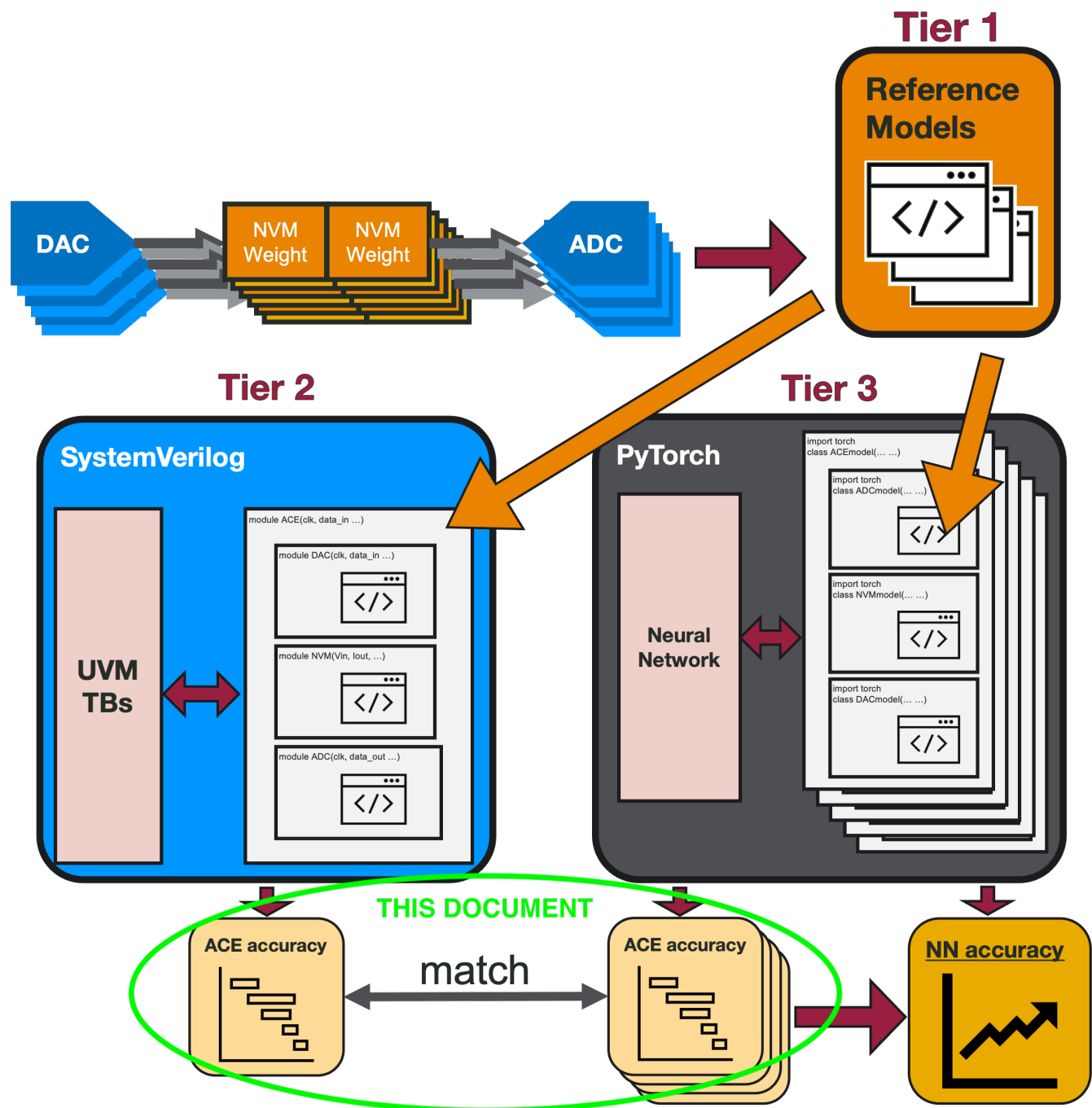
Mythic Modeling Process

The following diagram illustrates the modeling hierarchy described above:

- First, Mythic circuits team does extensive SPICE simulations to create the **reference models**
- SystemVerilog Real Number Models are created for the analog circuits that are built with the **same reference models** that allow digital simulation of an entire ACE calculation
- PyTorch uses the **same reference models** to run an entire network
- Mythic will confirm computational equivalency between the SystemVerilog computation and the PyTorch computation

MYTHIC

Figure 3. Modeling Process



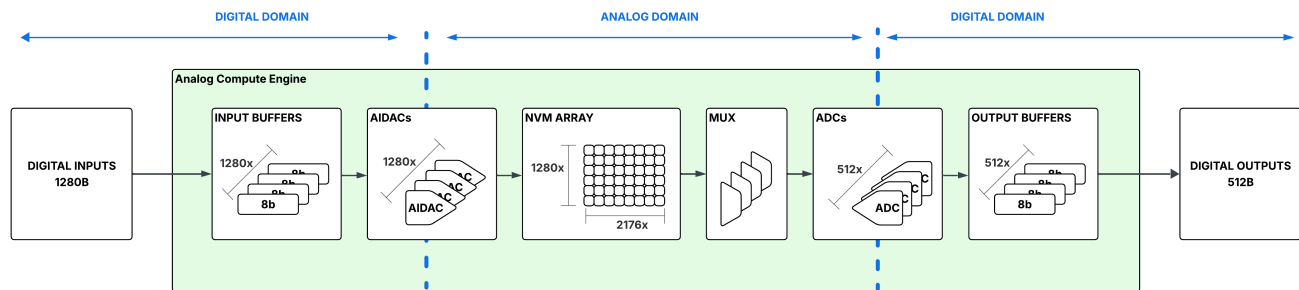
MYTHIC

2 Model Matching Discussion

2.1 Components Being Modeled

2.1.1 Signal Chain Diagram

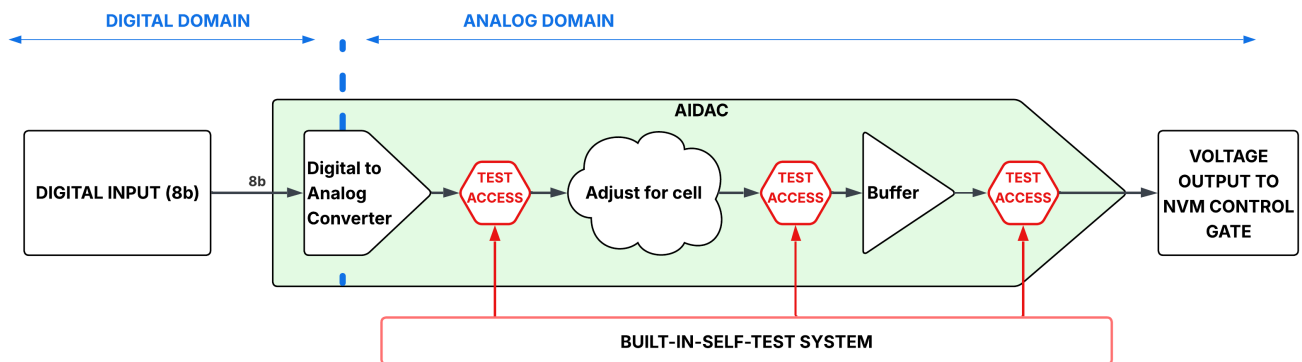
Below is the signal chain diagram, also shown in the Analog Compute Engine Operation document. **All components in the analog domain are included in this comparison.**



2.2 AIDAC Model

2.2.1 Diagram

Figure 4. AIDAC Diagram Showing Test Access



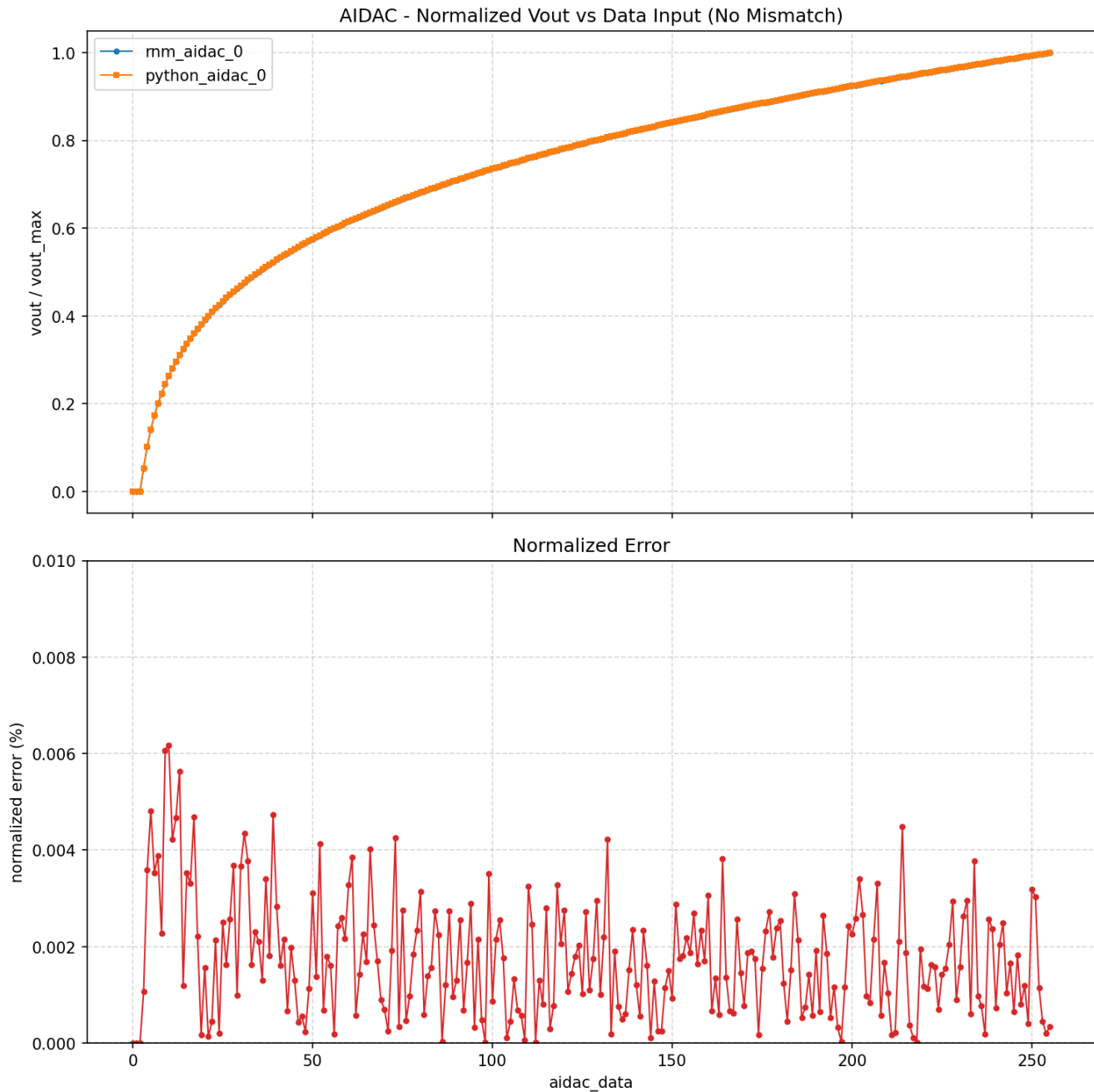
2.2.2 RNM vs. PyTorch Model Match Without Manufacturing Variation

First we present the matching results without individual element mismatch. Here we ramp the digital input word to the AIDAC and plot the normalized output voltage for both the RNM and PyTorch models. Beneath this sweep, we plot the error between the two in units of percent.

MYTHIC

As is shown, the error between the two versions of the model is less than 0.01 percent.

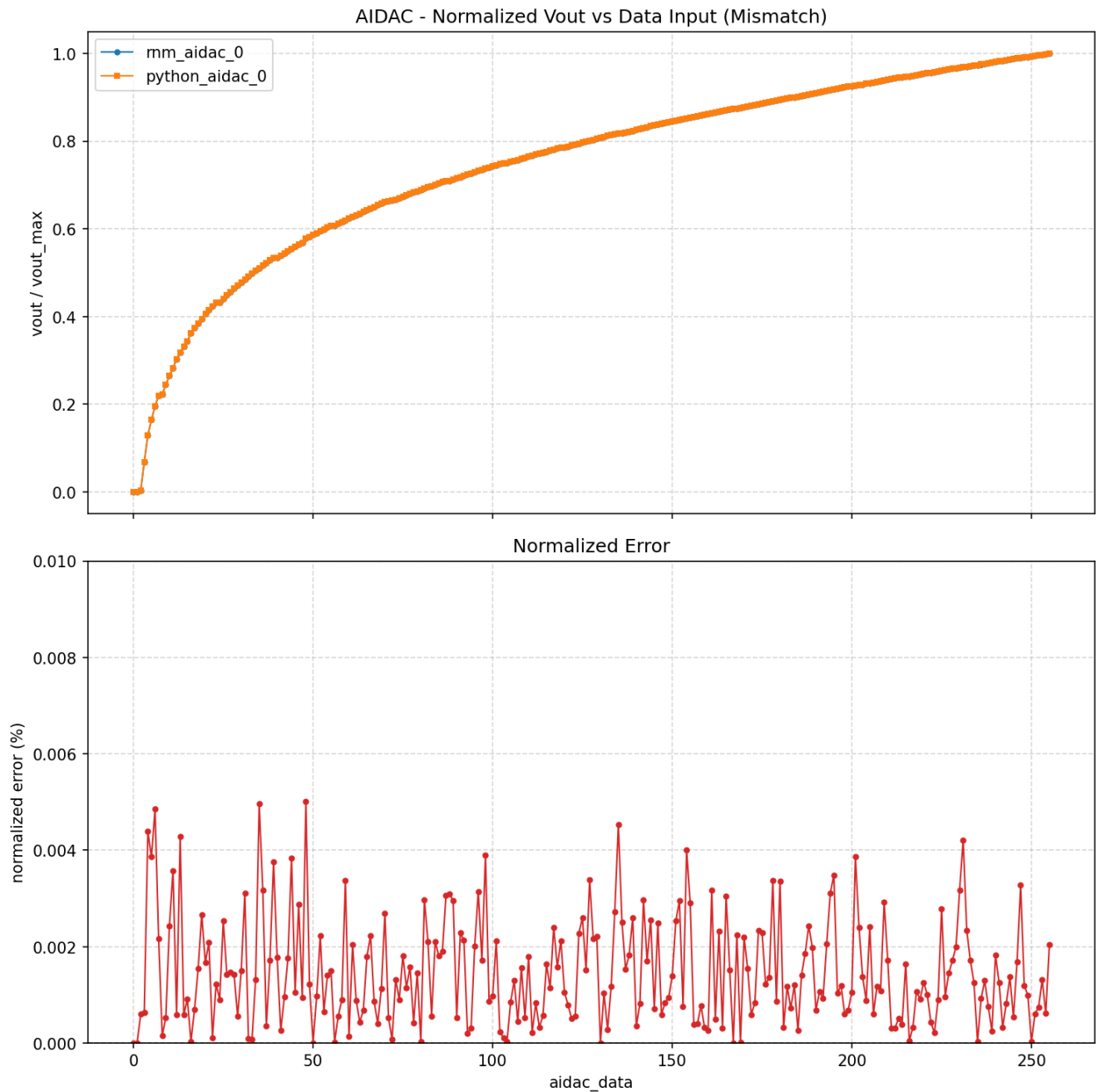
Figure 5. Normalized AIDAC Output and Error %, Without Element Mismatch



2.2.3 RNM vs. PyTorch Model Match With Identical Manufacturing Variation

Next, we enable individual element mismatch in both the RNM and PyTorch models. The error between the two versions of the model remains less than 0.01 percent.

Figure 6. Normalized AIDAC Output and Error %, With Element Mismatch



2.2.4 AIDAC Model Summary

We have achieved AIDAC model matching between the RNM and PyTorch with better than 0.01% error.

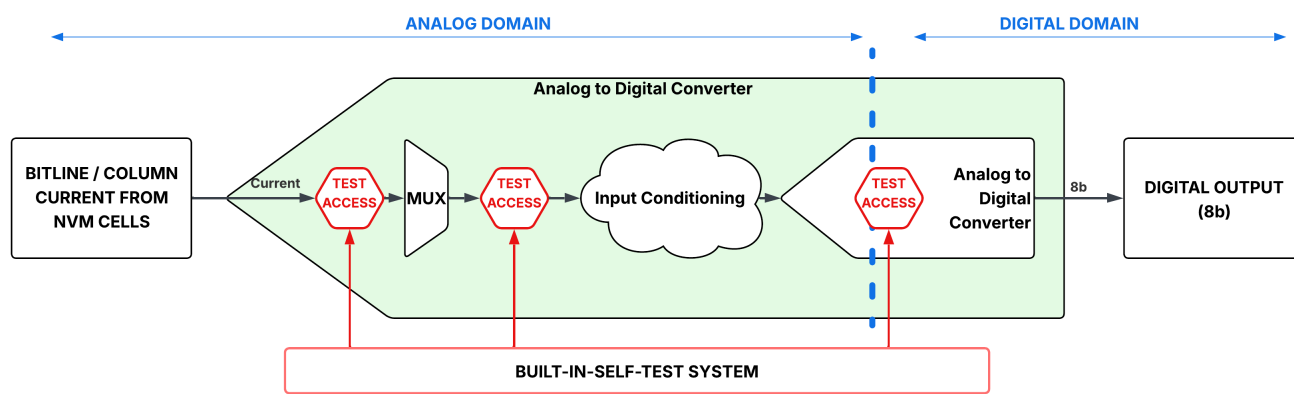
2.3 NVM Model

The NVM SPICE model is provided by SST (Microchip). This model is based on proprietary SST silicon measurements. Mythic performs a polynomial fit on the SPICE model simulation results and thus this polynomial is exactly the same in both PyTorch and RNM.

2.4 ADC Model

2.4.1 Diagram

Figure 7. ADC Diagram Showing Test Access



2.4.2 RNM vs. PyTorch Model Match Without Manufacturing Variation

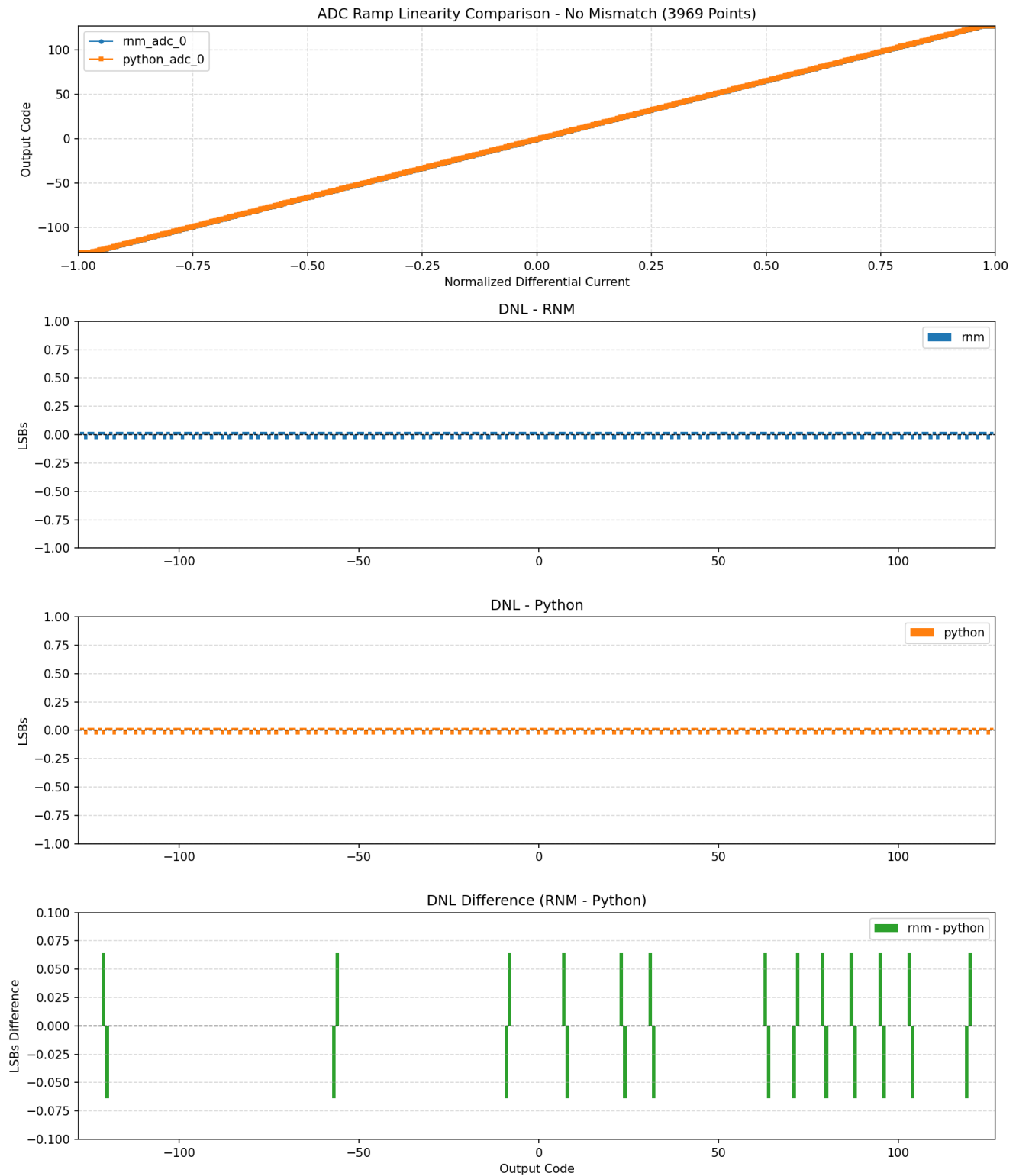
To compare nonlinearity in the ADC transfer function, we measure the ADC model Differential Non-Linearity (DNL). This will allow us to compare the width of each ADC code in each model.

To do this we send an input current ramp into the ADC with finer resolution than the ADC such that we have good measurement resolution of the ADC code widths. We use a ramp with 4,096 (12b) ramp steps, so in theory 16 ADC outputs per ADC code. However, to ensure we capture the full ADC range we over-range slightly. For this individual ADC with its individual configuration settings, this results in 3,969 ADC outputs between the transition from codes 0->1 and codes 254->255.

Between the minimum and maximum ADC codes we send in a ramp with 3,969 steps. DNL is extremely low, and is limited to the numerical artifact of dividing the 3,969 steps into 254 codes. This is expected because there is no element mismatch in the ADC to create significant non-linearity.

The only differences between RNM and PyTorch ADC models occur at code transition boundaries and are less than 0.075LSBs in magnitude.

Figure 8. ADC Output, DNL, and DNL Difference, Without Element Mismatch

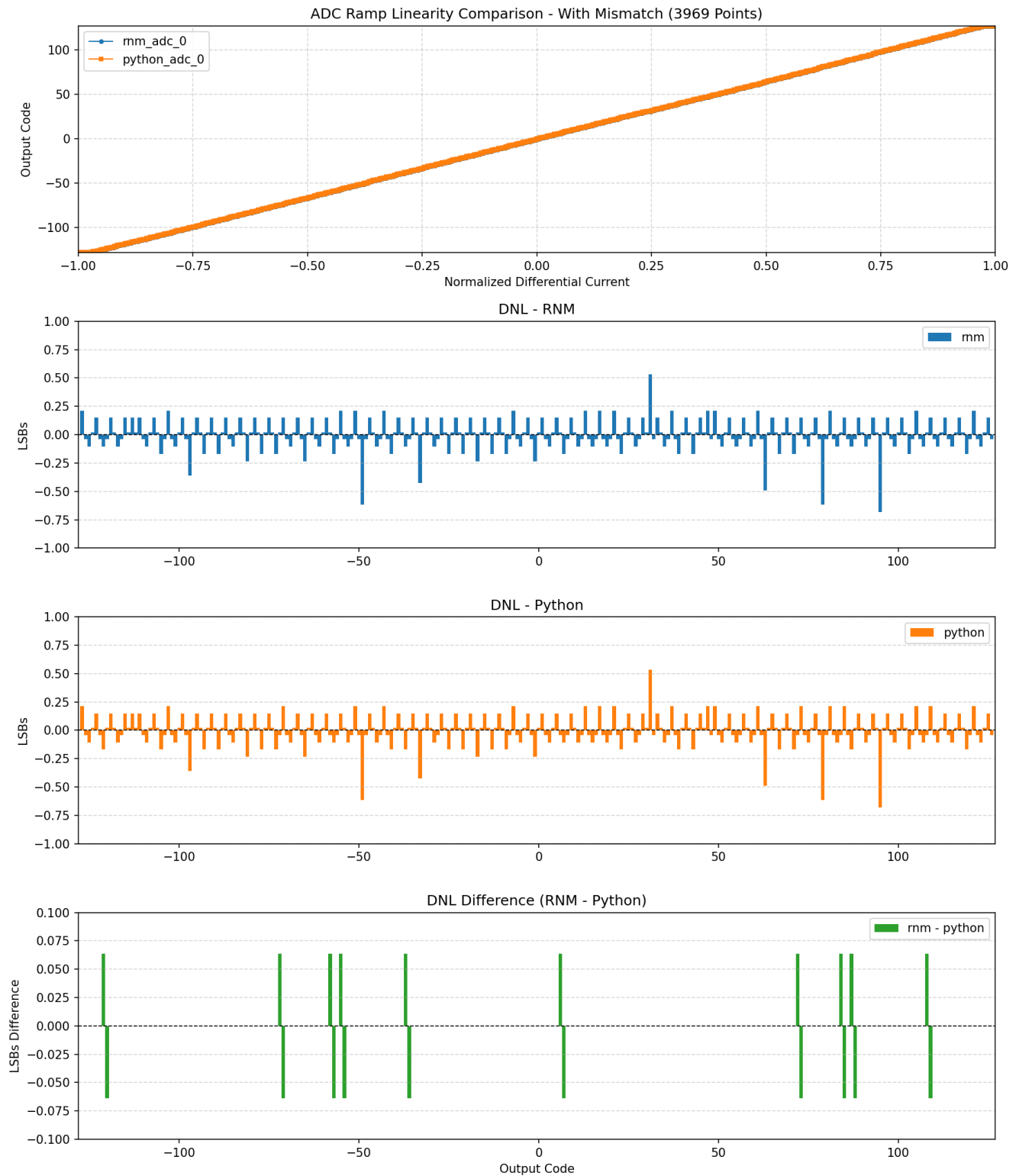


2.4.3 RNM vs. PyTorch Model Match With Identical Manufacturing Variation

Repeating the above ramp with identical element mismatch turned on, we now observe reasonable amounts of ADC non-linearity, with maximum DNL values of around $+0.5/-0.7$ LSBs.

Again, the only differences between RNM and PyTorch ADC models occur at code transition boundaries and remain less than 0.075LSBs in magnitude. This confirms the element mismatch contributing to DNL is identical.

Figure 9. ADC Output, DNL, and DNL Difference, With Element Mismatch



2.4.4 ADC Model Summary

We have achieved ADC model matching between the RNM and PyTorch within less than 0.1 LSBs of DNL.

3 Full ACE Model Comparison With Real Network Data

After determining the AIDAC and ADC models are equivalent, we now compare the RNM and PyTorch models of the entire ACE analog signal chain with real network data. This will show the combined effect of any errors throughout the signal chain.

Our test case is a Mythic-retrained layer from Yolov8 pose, with 1000 input vectors. Because this layer is mapped to 64 columns, this results in a total of 64,000 ADC outputs (or channel activations).

3.1 RNM vs. PyTorch Model Match Without Manufacturing Variation

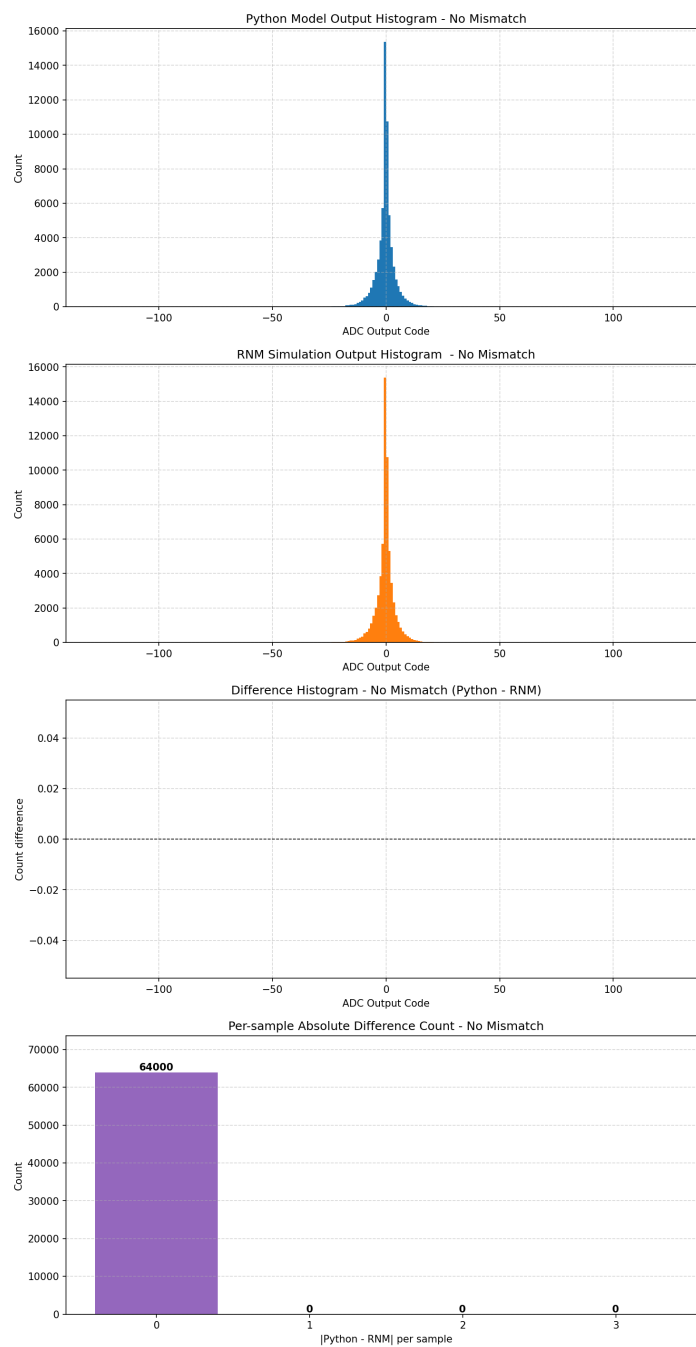
First we present the matching results without individual element mismatch. Each plot shows:

- A histogram of all 64,000 ADC output code values for the PyTorch model
- A histogram of all 64,000 ADC output code values for the RNM model
- A count of the differences between the two *for each individual ADC code*
- A count of the difference magnitudes in ADC codes

Here, for the ACE signal chain without element mismatch enabled, there is zero difference between any of the 64,000 ADC outputs in either model.

Figure 10. Full ACE Signal Chain Output, Without Element Mismatch

Yolov8 pose, conv_8 layer, 1000 input vectors



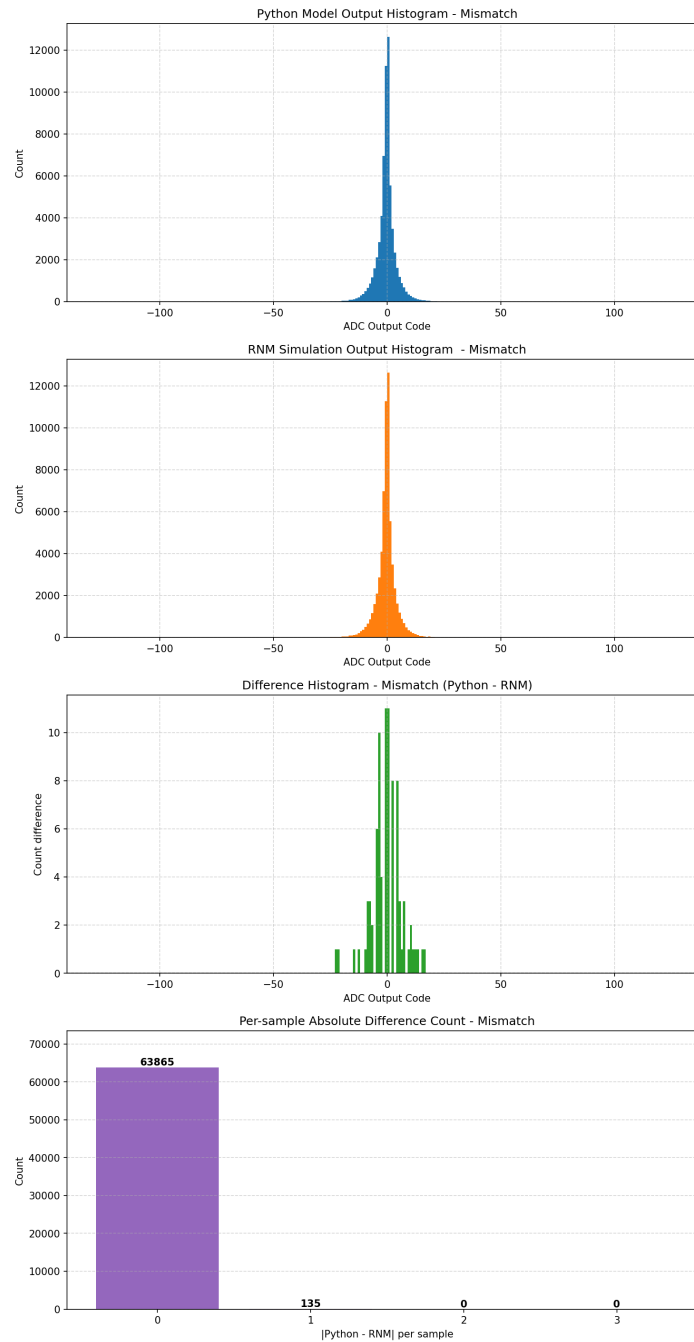
3.2 RNM vs. PyTorch Model Match With Manufacturing Variation

After enabling element mismatch, we now observe a very minor difference in output between the two models.

Note that:

- In all 64,000 outputs, only 135 outputs have any difference (**0.2%**)
- The magnitude of this output difference is never greater than 1 LSB

Figure 11. Full ACE Signal Chain Output, With Element Mismatch
Yolov8 pose, conv_8 layer, 1000 input vectors



3.3 Full ACE Model Comparison Summary

The output of the full ACE analog signal chain, in both RNM and PyTorch models, match within 0.2%, with the minimum error possible (1 ADC LSB).

4 Conclusion

ACE analog RNM model is ready to be used in DV / emulation. Any DV / emulation can now include analog ACE behavior.